第一阶段-STM32-阶段 3

25 年 11 月 17 日-25 年 11 月 23 日，STM32 串口通信、I2C 通信、SPI 通信

（严格要求了提交方式以及提交格式的，详细请看最下面）

<==========>

STM32—串口通信

简介：串口是一种通信模式，遵循一定的协议。串口是一种一对一的通信模式，一般情况下只需要对好波特率就可以通信。一般的模式有轮询，中断，DMA。
一般的器件回传的串口数据都是以数据包的形式呈现的，比如之前的 GPS 数据解析题目出现的那种，需要查询官方器件手册中的数据格式来解释数据包信息。
串口的通信线有两条，RX 和 TX。两个器件通信的电平维持时间由设置好的波特率决定。

基础任务：
1.使用 CH340 模块使单片机与电脑交互。实现最基本的回环交互信息；
2.使用任意蓝牙模块与手机连接，控制单片机点亮或熄灭小灯；
3.使用串口 DMA 接收不定长数据完成基础任务 1。

STM32—I2C 通信

简介：I2C 是一种常见的通信方式。I2C 最大的优点就是可以将多个同样遵循 I2C 协议的芯片挂载在同一个 I2C 总线下，节约单片机 IO 口资源。
一般的 I2C 都是对器件的寄存器进行操作，大多数的设置都是对指定寄存器写入某些值以达到你想要的效果，而这些值都可以通过查询器件的寄存器手册得到。
I2C 通信有两条，一般命名为 SCL 和 SDA。同一条 I2C 总线上的通信电平维持时间由 SCL（时钟线）决定，SDA 为数据线。SDA 一般为开漏，这样器件和主控都可以控制这根通信线来实现双向通信，所以它们不能同时发送信息。
一般 I2C 控制一个芯片需要先发送指定芯片的地址，芯片在接收到地址指令后会返回 ACK 告诉单片机收到消息准备开始接收命令了，这样就解决了总线上信息归属的问题。同时可以外部改变电平改变器件地址来实现同一总线上挂载两个相同器件。

基础任务：
1.学会使用软件 I2C，使用软件 I2C 驱动 OLED 屏幕；
2.学会使用 HAL 库的硬件 I2C，使用硬件 I2C 驱动 OLED 屏幕；
3.学习其他 I2C 器件，如陀螺仪等。

STM32—SPI 通信

简介：同 I2C，SPI 也是一种常见的通信方式，其也可以挂载多个器件成为 SPI 总线。不同于 I2C 的 SDA，SPI 一般有双向的通信线，也就是说它们可以同时发送或接收。
一般 SPI 的器件都是 DC（Data&Command）形式的。一个 Command 下可以写入不同 Data，或者直接一个单独的 Command。这也需要查询相关寄存器手册才能知到。

一般的 SPI 是 4 根线 MOSI（Master Out Slaver In）、MISO（Master In Slaver Out）、CS（片选信号）、SCK（时钟线）。比较特殊的器件还有 DC（Data&Command）、BUSY（忙碌）等。一般通信顺序是主控将 CS 拉低选中某个器件，SCK 搭配 MOSI 发送指令，然后从机通过 MISO 以相同的 SCK 返回信息，主机再将 CS 拉高关闭通信。也就是说每有一个 SPI 器件就需要一根 CS 线，而通信线是可以挂载在一起的。

基础任务：
1.学会使用硬件 SPI，SPI 用的少，一般是用在屏幕或一些通信模块中的，因为 SPI 比 I2C 快很多很多。遇到了再说吧。或者你可以试着写一写 LCD 屏幕，SPI 协议的。

<==========>

对于简单的学习，看看网上的教程然后跟着做，最后实现效果就行了。不会的就先思考再上网问。如果你好奇为什么的话就去深究。

<==========>

一般比较常见的通信方式就上面这三种的，比较无聊。但是都特别重要，因为很常见嘛。
简单说起来很简单，但是当你去做的时候，去研究每一个传入传出的值的时候，去对照寄存器表一个一个看的时候，才知道其复杂。
但是一上来就要求你们根据寄存器手册去完整写一个模块不太现实。**建议大家现阶段还是上网找库直接用**，因为呈现出来的效果最重要嘛。但是当你们到一定水平的时候我还是希望你们能够花时间下去研究每一个器件的寄存器级的操作设置，设置出最适合你们自己工程的样子。我很害怕以后大家都使用库级别的代码，而都不知道怎么写没有被写过的器件。只是片面的使用一款芯片最基础的功能而导致整个工程的冗余复杂，我认为这是对芯片设计者的最大亵渎。
所以在你们很后期的时候，我希望面对一个功能设计的时候，你们能够：选择大概芯片方向->同等芯片选型->查手册看推荐电路->根据寄存器手册写库->实现效果；同时关注芯片电气信息。

<==========>

有关代码风格，从现在起就要开始要求了。

这里简单的说一下。

1.#define 的东西需要全大写

例如：`#define MPU6500_REG_GYRO_CONFIG_FS_SEL_250      0x00`

2.函数名称最好是"主要对象_作用_对象"且每一块的第一个字母为大写

例如：`uint8_t MPU6500_Read_Accel(struct MPU6500_Data *mpu6500_data)`

3.全局变量名首单词字母小写，后面的单词首字母大写

例如：`nmeaTIME beiJingTime;`

4.局部变量名小写

例如：`HAL_StatusTypeDef res;`

5.Tab 的空格数为 4 个

6.注释风格

```
/*brief  读取加速度数据
 * @param  mpu6500_data: MPU6500 数据结构体指针
 * @retval 0:成功, 1:失败
 */
uint8_t MPU6500_Read_Accel(struct MPU6500_Data *mpu6500_data)
```

7.关于大括号

```
// 1.大括号对齐
// 大括号保证在同一列,包括 if,for 等大括号
int main(void)  // 空传参要写 void
{

    return 0;
}

// 这样看着很难受,不推荐
int main(void){

    return 0;
}

// 2.大括号省略
// 对于 if 和 for 等非函数大括号,如果只有一句话你可以提行不写大括号
if(a == b)
    return 0;

// 这样也可以
if(a == b)  return 0;

// 这样也可以
if(a == b)
{
    return 0;
}
```

比较重要的就这些了，后续还会根据情况添加。这个东西是约定俗称的，是为了大家之间代码看的更加清楚。当然这个也不是最规范的，也应该不存在最规范这一说。

<==========>

25 年 11 月 23 日（周天）下午两点到六点到实验室检查打分。带上你的成品电路和电脑，必要时可能会检查代码。